

In Class Practice

Question 1:

What is the time complexity to count the number of elements in the linked list? a.

O(1)

b. O(n)

c. O(logn)

d. O(nlogn)

Questions 2:

In circular linked list, insertion of node requires modification of.

a. One Pointer

b. Two Pointers

c. Three Pointers

d. No Pointer Modified

part e

Question 3: In your concatenated single-linked list, the output looks like the following answer:- (See code attached in Blackboard sll_concat)

First Linked List.. 1->15->40-

>45->

Second Linked List.. 5->14->30->

Concatenated List..

1->15->40->45->5->14->30->

Can you modify the following concat function to show the output looks like this:

Concatenated List..

5->14->30->1->15->40->45->

```
template<class T> void
```

```
Chain<T>::Concat(Chain<T> *a,Chain<T> *b)
```

```
{
```

```
    Node<T> *f, *s;
```

```
    f = a->first;
```

```
    s = b->first;
```

```
    Node<T> *p=f;
```

changing the first node being iterated through to f (the second list)

```
    while(p->link)
```

```
    {
```

```
        p = p->link;
```

```
    }
```

```
    p->link=s;
```

setting the end of the second list to link to the first node in the first list to connect them

```
    s=NULL;
```

together, then setting f to null since its last won't be connected to anything

```
    cout<<"\n\nConcatenated List..\n";
```

```
    Node<T> *temp=f; while(temp!=NULL)
```

change f to s so in the output it sets the temp pointer to s and the new connected list starts from the beginning, or the second list or s (see my pasted code for the changes)

```
    {
```

```
        cout<<temp->data<<"->";
```

```
        temp=temp->link;
```

```
    }
```

```
}
```

```
template<class T>
```

```
void Chain<T>::Concat(Chain<T> *a,Chain<T> *b)
```

```
{
```

```
    Node<T> *f, *s;
```

```
    f = a->first;
```

```
    s = b->first;
```

```
    Node<T> *p = s;
```

```
    while(p->link)
```

```
    {
```

```
        p = p->link;
```

```
    }
```

```
    p->link = f;
```

```
    f = NULL;
```

```
    cout << "\n\nConcatenated List..\n";
```

```
    Node<T> *temp = s;
```

```
    while(temp != NULL)
```

```
    {
```

```
        cout << temp->data << "->";
```

```
        temp = temp->link;
```

**Don't just change f or s variable to get the expected output